Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

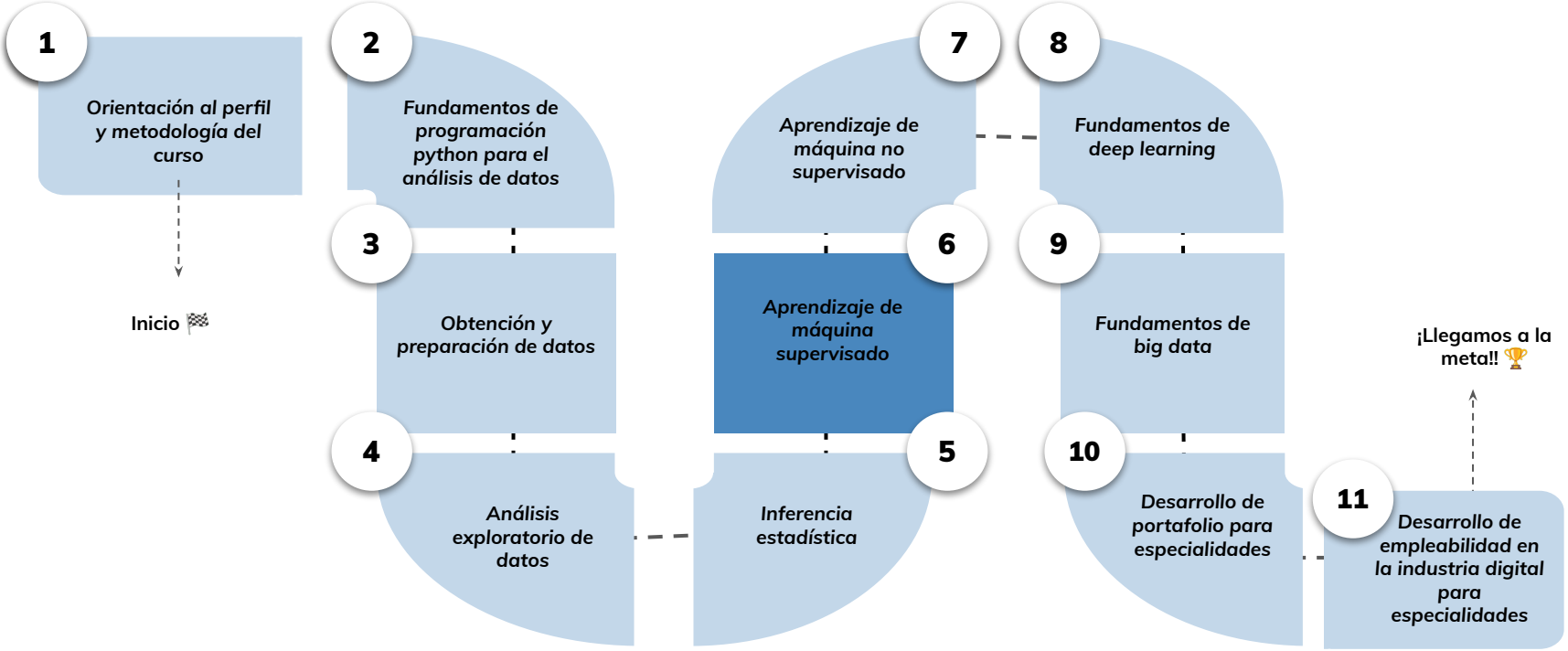
Nivel de ajuste de un modelo y validación cruzada- Parte II

Aprendizaje Esperado 2: Aplicar técnicas de validación cruzada para la prevención de sobreajuste en los modelos utilizando el concepto de nivel de ajuste óptimo.

alkemy

Hoja de ruta

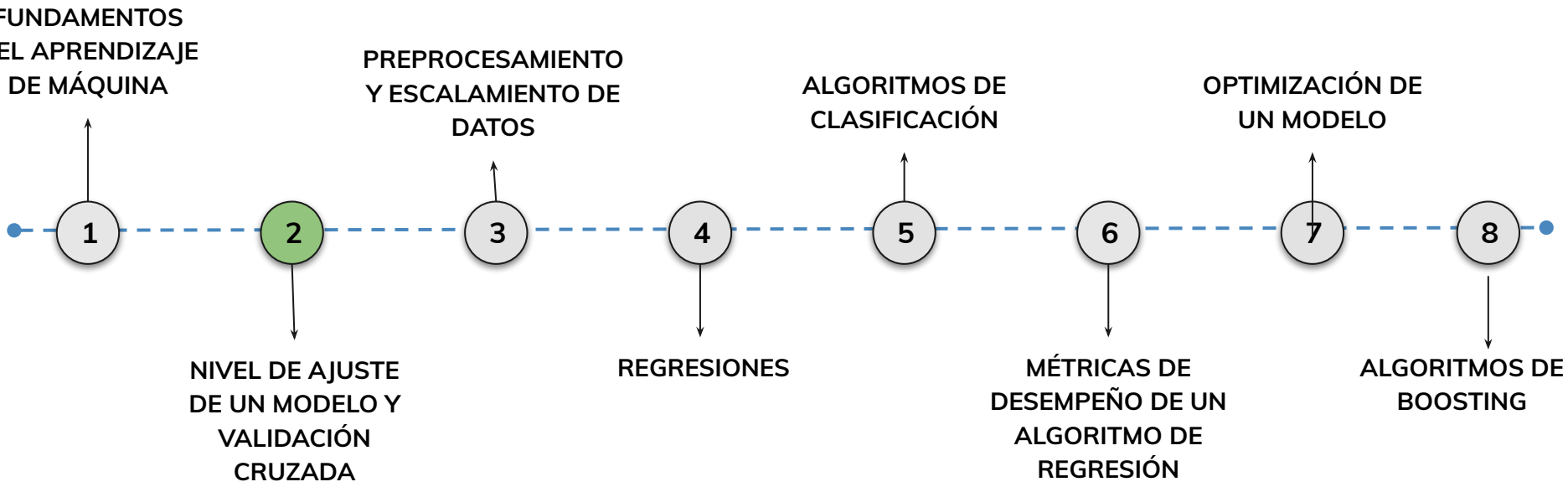
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

2.

Tipos de validación cruzada

Conoceremos en profundidad las variantes más comunes de validación cruzada, sus ventajas, desventajas y casos de uso.

- **K-Fold, LOOCV, ShuffleSplit**
  - **Comparativa entre métodos**
  - **Cuándo aplicar cada uno**
- **Implementación práctica con Scikit-learn**
  - **Uso de cross_val_score y clases específicas**
  - **Interpretación de resultados y decisiones**

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- Aplicamos diferentes técnicas de validación cruzada
- Comparamos K-Fold, LOOCV y ShuffleSplit
- Implementamos validación cruzada con Scikit-learn
- Identificamos cuándo usar cada técnica
- Evaluamos modelos de forma robusta y replicable

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Subajuste, sobreajuste y ajuste adecuado*

→ *Error por sesgo y varianza*

→ *Evaluación con train_test_split*

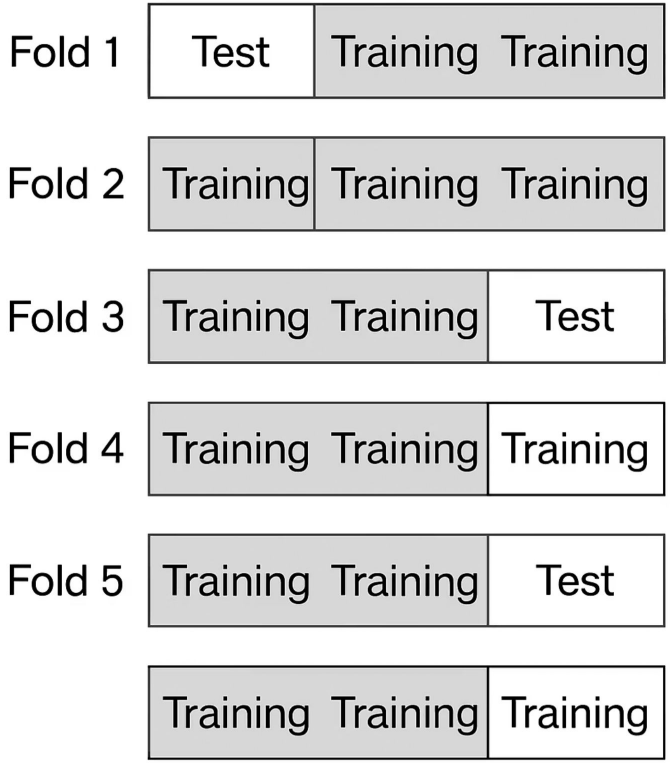
→ *Introducción a la validación cruzada (K-Fold)*

Nivel de ajuste de un modelo y validación cruzada

Validación Cruzada de K-Iteraciones

La validación cruzada de k-iteraciones, también conocida como **K-Fold Cross Validation**, es una técnica sistemática que mejora la estimación de desempeño de un modelo y reduce el sesgo asociado a una única partición de los datos.

k-Fold Cross-Validation



alkemy

Proceso de K-Fold Cross Validation

Esta técnica consiste en dividir el conjunto de datos en *k* subconjuntos o "folds" de tamaño similar. En cada iteración, uno de estos subconjuntos se utiliza como conjunto de prueba y los *k-1* restantes como conjunto de entrenamiento. Este proceso se repite *k* veces, de forma que cada subconjunto haya sido utilizado exactamente una vez como conjunto de prueba.

Configuración de K-Fold

Valor de k

El valor de *k* es configurable, aunque los más comunes son 5 o 10.

Valor pequeño de k

Un valor pequeño de *k* puede generar una alta varianza en la evaluación.

Valor grande de k

Un valor muy grande puede aumentar el coste computacional.

Beneficios de K-Fold Cross Validation

Independencia de la partición

El principal beneficio de la validación cruzada de k-iteraciones es que reduce la dependencia de la evaluación respecto a un único conjunto de prueba.

Aprovechamiento de datos

Además, permite aprovechar mejor el conjunto de datos, ya que cada observación es utilizada tanto para el entrenamiento como para la evaluación.

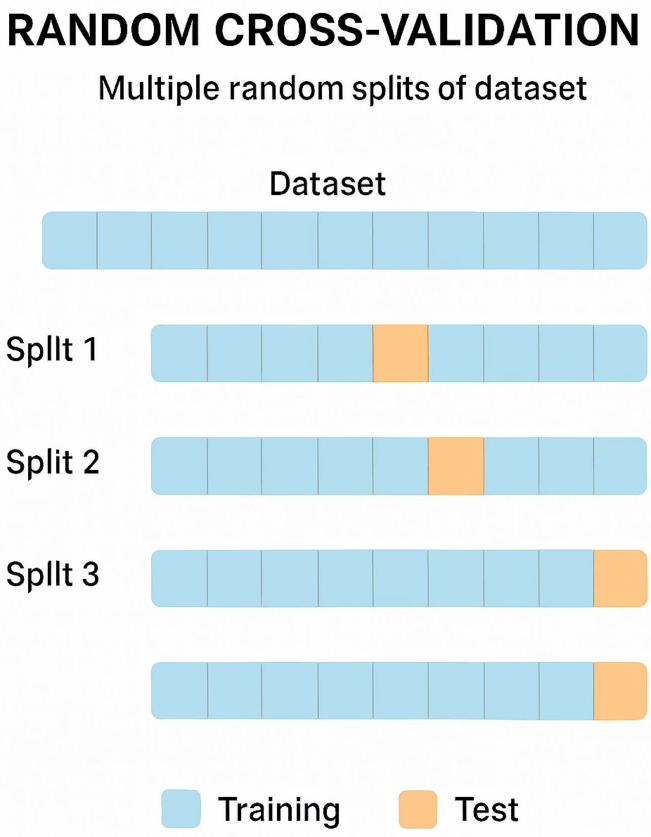
Desventajas de K-Fold Cross Validation

Entre sus desventajas, destaca el incremento en el tiempo de entrenamiento, ya que el modelo debe ser entrenado y evaluado *k* veces. Esto puede ser un factor limitante para modelos complejos o conjuntos de datos muy grandes.

A pesar de ello, es considerada una de las técnicas estándar y más fiables para evaluar modelos de Machine Learning, proporcionando un balance adecuado entre sesgo y varianza en la estimación de rendimiento.

Validación Cruzada Aleatoria

La validación cruzada aleatoria, también conocida como **Shuffle Split** o particionado aleatorio repetido, es una técnica que busca evaluar el desempeño del modelo mediante múltiples particiones aleatorias de los datos.



alkemy

Funcionamiento de Shuffle Split

En este enfoque, se especifica un porcentaje para el conjunto de entrenamiento y otro para el conjunto de prueba. La división se realiza aleatoriamente varias veces, y el rendimiento final del modelo se obtiene promediando los resultados de todas las particiones.

Ventajas de la validación cruzada aleatoria

Flexibilidad

Una de las ventajas clave de este método es la flexibilidad que ofrece al permitir controlar tanto el tamaño de los subconjuntos como la cantidad de particiones.

Eficiencia con datos grandes

Esto es útil para conjuntos de datos grandes, donde no es necesario utilizar todos los datos en cada iteración.

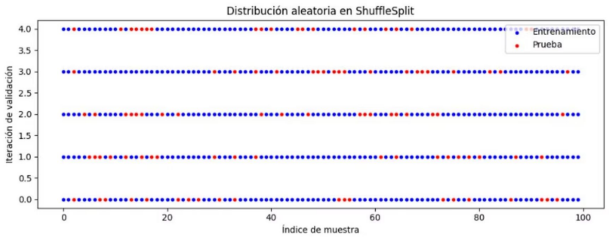
Desventajas de la validación cruzada aleatoria

Uso desigual de datos

Puede darse el caso de que ciertos datos queden sin ser utilizados, o que algunos se repitan con frecuencia en diferentes particiones, lo que introduce cierto grado de dependencia entre las muestras.

No garantiza cobertura total

Otra limitación es que no garantiza que todas las observaciones hayan sido utilizadas para la evaluación, a diferencia de la validación cruzada de k-iteraciones.



🧪 Ejemplo de código con `ShuffleSplit` y `cross_val_score`

```
from sklearn.model_selection import ShuffleSplit, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
import numpy as np

# Dataset ficticio
X, y = make_classification(n_samples=500, n_features=10, random_state=42)

# Configurar validación cruzada aleatoria
shuffle = ShuffleSplit(n_splits=5, test_size=0.25, random_state=0)

# Modelo
modelo = LogisticRegression()

# Evaluar con cross_val_score
scores = cross_val_score(modelo, X, y, cv=shuffle, scoring='accuracy')

print("Scores por partición:", scores)
print("Precisión promedio:", np.mean(scores))
```

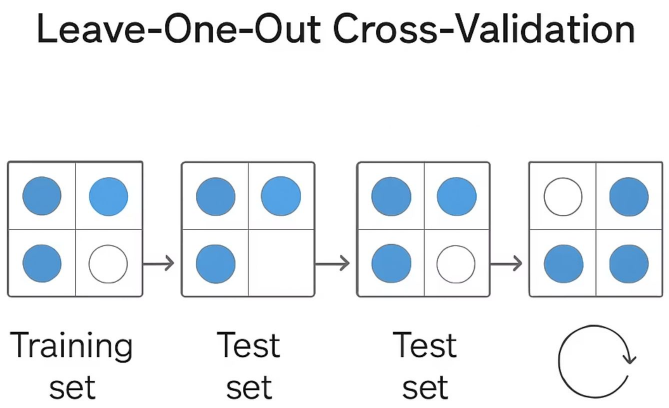
Aplicaciones recomendadas

A pesar de estos aspectos, esta técnica resulta eficiente y recomendable cuando el volumen de datos es considerable y el costo computacional de una validación exhaustiva resulta prohibitivo.

alkemy

Validación Cruzada Dejando Uno Afuera

La validación cruzada dejando uno afuera, conocida como **Leave-One-Out Cross-Validation (LOOCV)**, es una técnica extrema de validación cruzada donde el número de iteraciones es igual al número de observaciones en el conjunto de datos.



alkemy

Proceso de LOOCV

En cada iteración, se utiliza una sola observación como conjunto de prueba y el resto como conjunto de entrenamiento. Este proceso se repite para cada observación, asegurando que todas sean utilizadas como prueba una vez.

Beneficios de LOOCV

Maximización de datos de entrenamiento

El principal beneficio de LOOCV es que maximiza el uso de los datos para el entrenamiento, minimizando el sesgo en la evaluación.

Evita problemas de particionado

Además, no requiere dividir arbitrariamente el conjunto de datos, evitando así problemas de particionado.

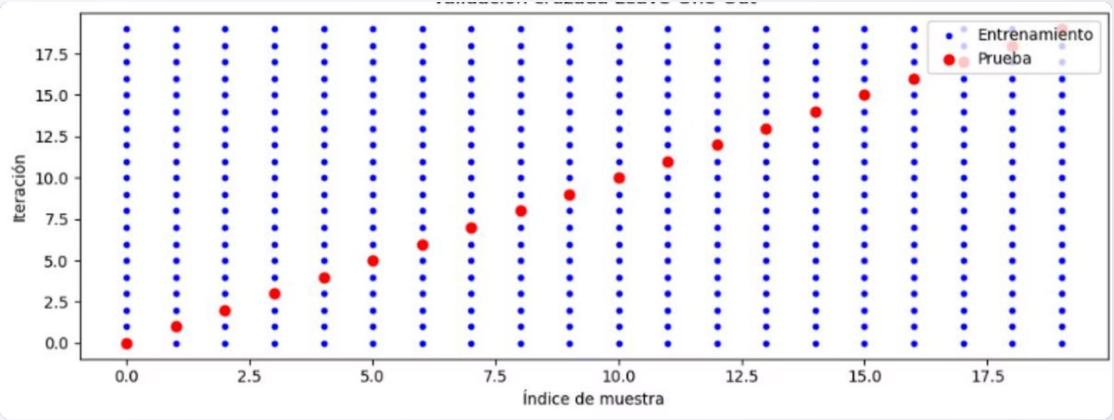
Desventajas de LOOCV

Alto coste computacional

Su desventaja principal es el coste computacional, ya que requiere entrenar el modelo tantas veces como observaciones haya. Para conjuntos de datos grandes o modelos complejos, esto puede ser inviable.

Alta varianza en la estimación

Además, la varianza de la estimación de error puede ser alta, ya que cada modelo es validado sobre un solo dato, que puede ser atípico o poco representativo.



Aplicaciones de LOOCV

A pesar de ello, LOOCV es útil en contextos académicos o cuando se trabaja con conjuntos de datos pequeños, donde maximizar el uso de los datos es crucial.

alkemy

Ejemplo práctico con LeaveOneOut

Este código muestra cómo implementar la validación cruzada dejando uno afuera utilizando la librería Scikit-Learn.

```python
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
import numpy as np

# Dataset pequeño
iris = load_iris()
X, y = iris.data, iris.target

# Seleccionamos solo dos clases para binarizar
X, y = X[y != 2], y[y != 2]

# Configuración LOOCV
loo = LeaveOneOut()

# Modelo
modelo = LogisticRegression(max_iter=1000)

# Evaluación
scores = cross_val_score(modelo, X, y, cv=loo, scoring='accuracy')
print("Precisión promedio con LOOCV:", np.mean(scores))
```

Implementación de Validación Cruzada con la Librería Scikit-Learn

Scikit-Learn, una de las bibliotecas más utilizadas en Machine Learning con Python, ofrece herramientas prácticas y eficientes para implementar las distintas técnicas de validación cruzada.

```
from sklearn_model_selecti
import cross_val_score

scores = cross_val_score(
  classifier, X, y == 5)
  print(scores
```

alkemy

Herramientas para el método de retención

Para el método de retención, Scikit-Learn dispone de la función train_test_split, que permite dividir el conjunto de datos de forma sencilla, especificando el tamaño del conjunto de prueba y un estado aleatorio para reproducibilidad.

```
from sklearn.model_selection import train_test_splitX_train, X_test, y_train, y_test = train_test_split( X, y,
test_size=0.3, random_state=42)
```

Herramientas para K-Fold Cross Validation

La validación cruzada de k-iteraciones puede implementarse fácilmente mediante la clase KFold o la función cross_val_score. Estas herramientas permiten definir el número de particiones y realizar la evaluación automática del modelo, devolviendo las métricas correspondientes.

```
from sklearn.model_selection import KFold, cross_val_scorekf = KFold(n_splits=5, shuffle=True,
random_state=42)scores = cross_val_score(model, X, y, cv=kf)
```

alkemy

Herramientas para validación cruzada aleatoria

La validación cruzada aleatoria está disponible mediante la clase ShuffleSplit, que permite definir tanto el número de divisiones como el tamaño relativo de los subconjuntos de entrenamiento y prueba, proporcionando gran flexibilidad.

```
from sklearn.model_selection import ShuffleSplitss = ShuffleSplit(n_splits=10, test_size=0.25,
random_state=42)scores = cross_val_score(model, X, y, cv=ss)
```

Herramientas para LOOCV

Para la validación dejando uno afuera, Scikit-Learn ofrece la clase LeaveOneOut, que genera los índices necesarios para dividir los datos siguiendo esta técnica. También puede integrarse con cross_val_score para automatizar la evaluación.

```
from sklearn.model_selection import LeaveOneOutloo = LeaveOneOut()scores = cross_val_score(model, X, y, cv=loo)
```

```python
from sklearn.model_selection import cross_val_score, KFold
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)
model = DecisionTreeClassifier()
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kfold)

print(f"Scores: {scores}")
print(f"Mean Accuracy: {scores.mean():.2f}")
```

Ejemplo práctico básico de K-Fold Cross Validation

Este código muestra cómo implementar la validación cruzada de k-iteraciones utilizando Scikit-Learn.

alkemy

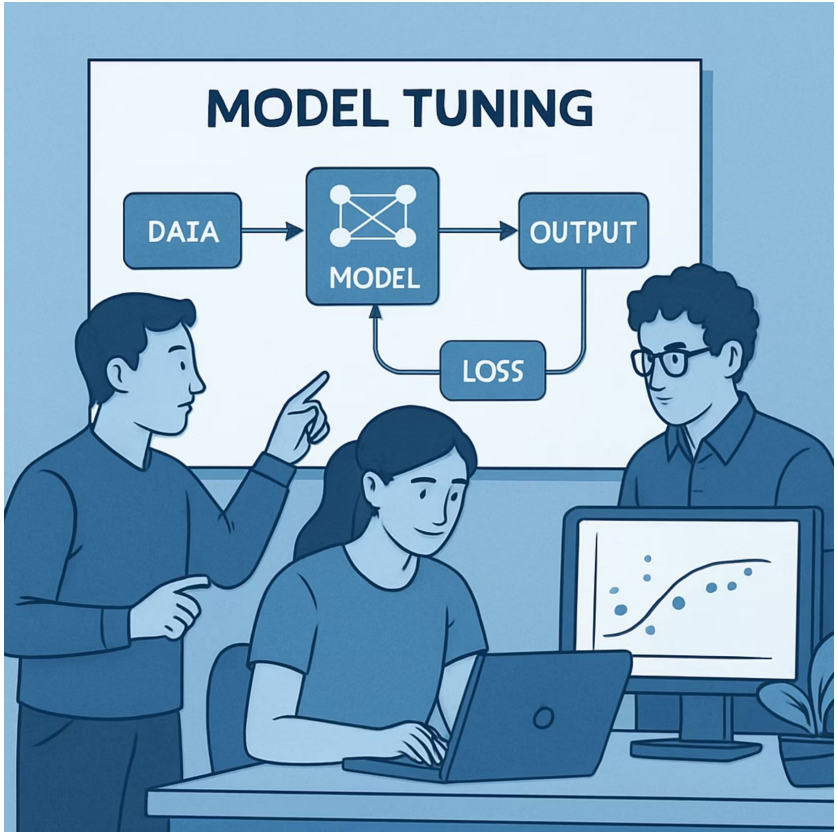
Integración práctica

Esta integración práctica permite evaluar de manera sistemática el rendimiento de los modelos, facilitando la comparación entre ellos y mejorando la toma de decisiones durante el proceso de modelado.

alkemy

Importancia del nivel de ajuste

En el desarrollo de un modelo de aprendizaje automático, comprender y gestionar el nivel de ajuste es fundamental para garantizar resultados precisos y generalizables.



alkemy

Evaluación correcta de errores

La correcta evaluación de los errores y el análisis del comportamiento del modelo frente a los datos de entrenamiento y prueba permiten identificar situaciones de sobreajuste, subajuste o ajuste adecuado, aspectos clave para la toma de decisiones informadas durante el proceso de modelado.

Validación cruzada como herramienta indispensable

La validación cruzada emerge como una herramienta indispensable para medir la capacidad de generalización de los modelos. A través de diferentes técnicas, como el método de retención, la validación cruzada de k-iteraciones, la validación cruzada aleatoria y la validación dejando uno afuera, es posible obtener estimaciones más robustas y confiables del desempeño de los modelos, evitando que las evaluaciones dependan exclusivamente de una única partición de los datos.

Comparación de técnicas de validación cruzada

1

Hold-Out

Simple y rápido, pero sensible a la partición específica de los datos.

2

K-Fold

Balance entre costo computacional y aprovechamiento de datos. Estándar en la industria.

3

Shuffle Split

Flexible y eficiente para conjuntos grandes, pero puede dejar datos sin usar.

4

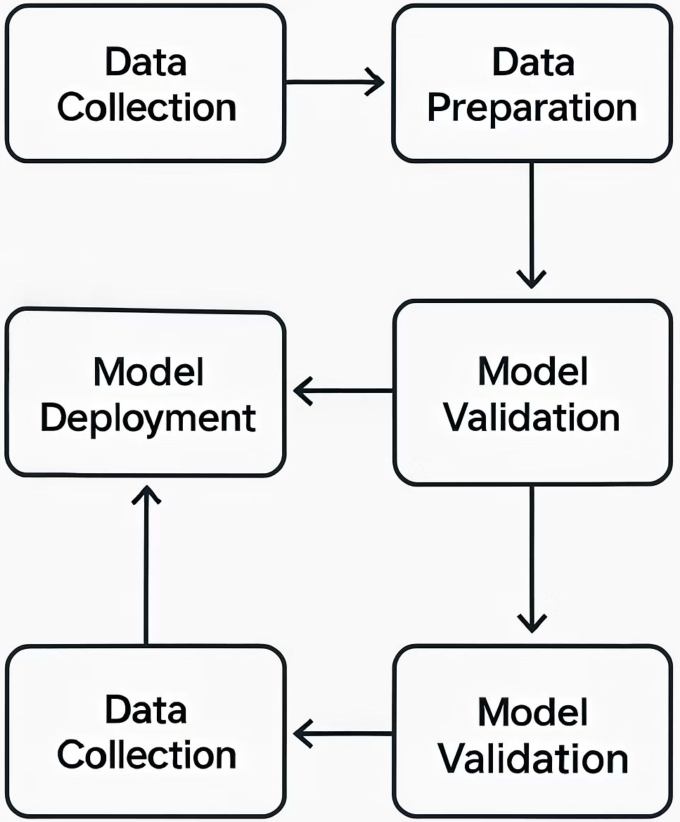
LOOCV

Máximo aprovechamiento de datos, pero computacionalmente costoso.

alkemy

Integración en flujos de trabajo

Finalmente, la integración de estas técnicas en flujos de trabajo prácticos mediante librerías como Scikit-Learn facilita la implementación de procesos rigurosos de validación y comparación de modelos.



alkemy

Aplicación en entornos de producción

La correcta aplicación de estas técnicas permite a los profesionales de datos construir modelos eficientes, capaces de resolver problemas reales y adaptarse adecuadamente a los desafíos del entorno de producción.

Conclusión

1

Ajuste adecuado

El nivel de ajuste de un modelo es crucial para su capacidad de generalización y debe ser cuidadosamente evaluado.

2

Validación robusta

La validación cruzada proporciona estimaciones más confiables del rendimiento real del modelo que una simple división de datos.

3

Selección de técnica

La elección de la técnica de validación debe considerar el tamaño de los datos, la complejidad del modelo y los recursos disponibles.

4

Implementación práctica

Herramientas como Scikit-Learn facilitan la implementación de estas técnicas en flujos de trabajo reales de ciencia de datos.

alkemy

Live Coding

¿En qué consistirá la Demo?

Vamos a aplicar distintas técnicas de validación cruzada sobre el mismo modelo y dataset, para comparar sus resultados.

1. *Cargar datos (e.g., load_diabetes o dataset propio)*
2. *Separar X e y*
3. *Configurar 3 validaciones: KFold, ShuffleSplit, LeaveOneOut*
4. *Ejecutar cross_val_score con cada una*
5. *Mostrar y comparar métricas*
6. *Graficar las distribuciones si el tiempo lo permite*
7. *Discutir ventajas y limitaciones prácticas*
8. *Conclusión: ¿cuándo conviene cada técnica?*

Tiempo: 25 Minutos

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio

Comparando técnicas de validación cruzada

Comparando técnicas de validación cruzada

Contexto: 🙌

Estás desarrollando un modelo de regresión para predecir el precio de alquiler de propiedades. Quieres evaluar su desempeño de forma robusta y confiable antes de implementarlo en producción.

Consigna: ✍️

Usando el dataset de viviendas (e.g., AmesHousing, Boston, Airbnb, o uno propio), compare el desempeño del mismo modelo utilizando tres técnicas distintas de validación cruzada:

- K-Fold (k=5)
- ShuffleSplit (n_splits=10, test_size=0.25)
- LeaveOneOut

Paso a paso: ⚙️

1. Cargue y preparar el dataset
2. Normalice los datos si es necesario
3. Creae un modelo de regresión (LinearRegression, DecisionTree.)
4. Aplique las tres técnicas de validación
5. Guarde y compare los resultados (R^2 y MAE promedio)
6. Reflexionar:
   → ¿Que técnica mostró mayor estabilidad?
   → ¿Cuál tuvo mayor costo computacional?
   → ¿Cuál usarías en producción?

Tiempo 🕐**:** *45 Minutos*

alkemy

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Aplicamos múltiples técnicas de validación cruzada***
- ✓ ***Implementamos K-Fold, ShuffleSplit y LOOCV con Scikit-learn***
- ✓ ***Comparamos métricas de evaluación en distintos contextos***
- ✓ ***Reflexionamos sobre cuándo usar cada enfoque***
- ✓ ***Ganamos experiencia para seleccionar técnicas robustas en proyectos reale***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy